SERVER MONITORING DATA · 28 OCT 2026

What this monitoring data can do with machine learning

16 Grafana panels from one Docker host running the iMOPS Node.js API. Each page below shows one chart, what it says, and the machine learning idea it supports.

90.3% disk used, still rising	**~4.4 days** until disk is full at current trend	**77.9–87.1%** host RAM used over 12 h
1 restart event (22:17)	**4.6 h** longest gap with no data	**89/89** health checks UP

00 · THE DATA

What is in the export

DASHBOARD	PANELS	SAMPLING	TIME COVERED
Host / Docker dashboard	Host CPU, Host RAM, Root disk, Network receive	every 2 min	10:36 – 22:36 (12 h)
Host / Docker dashboard	Container CPU, Container memory (/docker)	every 1 min	10:37 – 22:37 (12 h)
iMOPS app dashboard	Host CPU, RAM, disk, network inbound	every 15 s	22:17 – 22:38 (~20 min)
iMOPS app dashboard	Node.js event loop lag, process memory (RSS)	every 15 s	22:17 – 22:38 (~20 min)
iMOPS app dashboard	API request rate, P99 latency, HTTP 200 rate, availability	every 15 s	22:17 – 22:39 (~20 min)

IN ONE SENTENCE

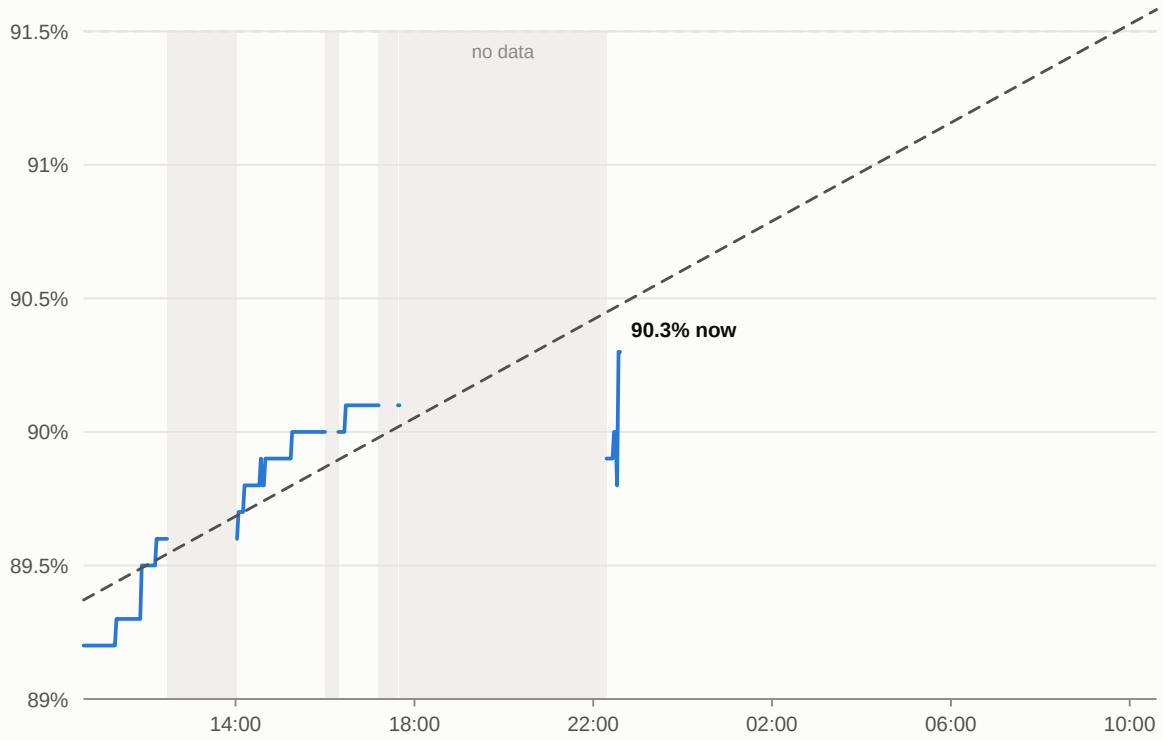
Two Grafana dashboards: a 12-hour view of the server and its Docker containers, and a 20-minute close-up of the iMOPS API that starts at the moment the containers restarted.

THE FIVE MACHINE LEARNING IDEAS IN THIS DECK

1. Predict when the disk will be full
2. Spot unusual behaviour automatically
3. Warn before an outage happens
4. Tell a planned restart from a real failure
5. Plan capacity and right-size the server

01 · USE CASE 1 · FORECASTING

The disk is 90% full and filling up



Solid line: measured disk use. Dashed line: straight-line trend fitted to all 157 readings, extended 12 hours ahead.

WHAT THE CHART SHOWS

- Disk use rose from 89.2% to 90.3% in 12 hours.
- The fitted trend is about +0.09 percentage points per hour.
- At that rate the disk reaches 100% in about **4.4 days** (around Mon 02 Nov).
- Grey bands are times with no data.

THE IDEA

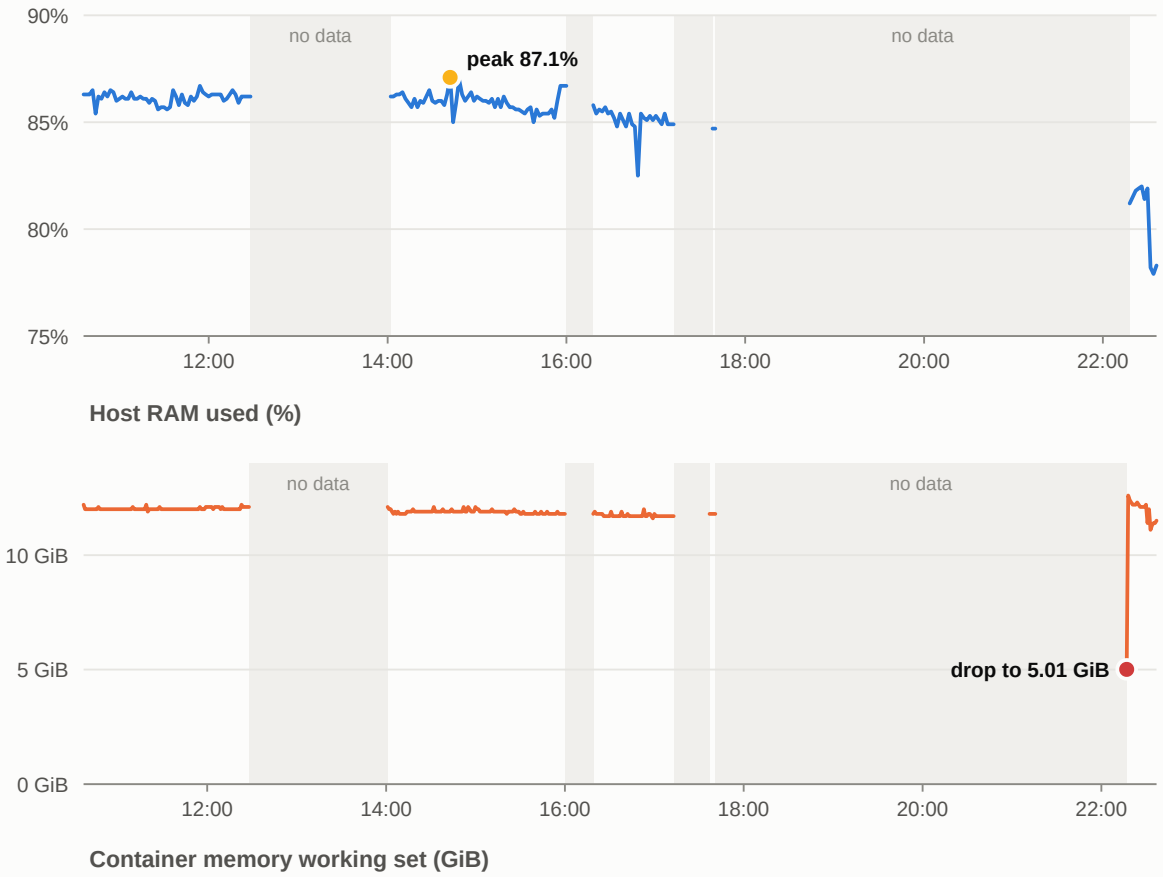
> A **forecasting model** projects disk growth and tells the team the date the disk will be full, so they clean up or add space before Docker, logs and databases stop working.

WHY THIS DATA WORKS FOR IT

- The trend is steady and mostly in one direction, which is the easiest kind of series to forecast.
- A full disk is one of the most common causes of server outages, so the warning is worth a lot.
- With weeks of history the model can also learn daily patterns, such as log rotation or backups.

02 · USE CASE 3 · EARLY WARNING

Memory sits high for hours



WHAT THE CHART SHOWS

- Host RAM stayed at about 86% from morning until the data stopped at 17:40.
- After the 22:17 restart it settled lower, around 77.9–82%.
- Containers use about 11.9 GiB on average. At 22:17 memory dropped to 5.01 GiB and came straight back.
- Memory that stays this high is a common warning sign before a crash.

THE IDEA

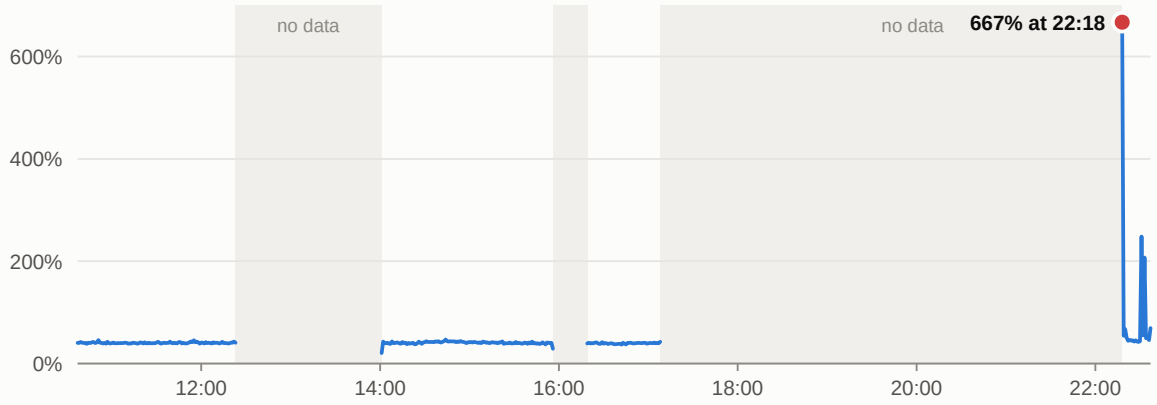
> An **early-warning model** learns the readings that come before a crash, such as RAM staying high for hours, and raises a risk score 15 to 30 minutes ahead instead of after the service is down.

WHY THIS DATA WORKS FOR IT

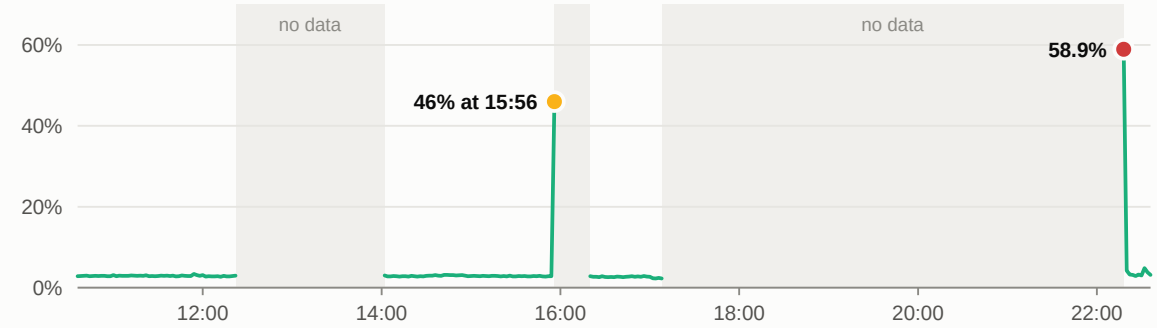
- Memory pressure builds slowly, so there is time to warn before it fails.
- The restart at 22:17 is a clear event the model can learn from.
- To train it reliably you need weeks of data that include several real incidents.

03 · USE CASE 2 · ANOMALY DETECTION

Spikes stand out against a quiet baseline



Container CPU (% of one core; 100% = one full core)



Host CPU busy (%)

WHAT THE CHART SHOWS

- Normally the containers use about 40% (under half a core) and the host about 3.6%.
- At 22:18 container CPU jumped to 667% (almost 7 cores) and host CPU to 58.9%.
- Both returned to normal within about 2 minutes.
- Host CPU also spiked to 46% at 15:56, right before the data stopped at 16:00. A spike just before a gap is exactly the warning sign an outage model looks for.

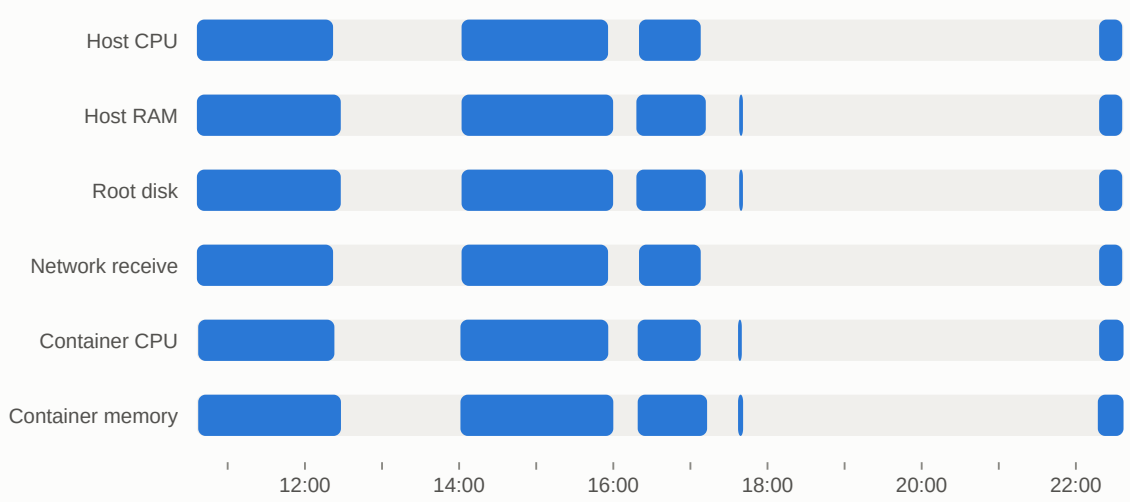
THE IDEA

> An **anomaly-detection model** learns what normal looks like across CPU, memory and lag together, and flags anything unusual, including combinations nobody wrote an alert rule for.

WHY THIS DATA WORKS FOR IT

- The baseline is very stable, so unusual readings stand out clearly.
- Several metrics move at the same moment, which is what multi-metric models are good at.
- It needs no labels: the model learns normal behaviour from the data itself.

Where the monitoring went silent



Blue: data received. Grey: no data. The iMOPS dashboard only starts at 22:17, after the longest gap.

GAPS IN THE HOST DATA

- 12:28 → 14:02 (94 min)
- 16:00 → 16:18 (18 min)
- 17:12 → 17:38 (26 min)
- 17:40 → 22:18 (278 min)

WHAT THE CHART SHOWS

- All six 12-hour panels go silent at the same times, so the whole host or its metrics collector stopped.
- The longest gap is about 4.6 hours (17:40 → 22:18).
- When the data comes back at 22:17, the containers have just restarted.

THE IDEA

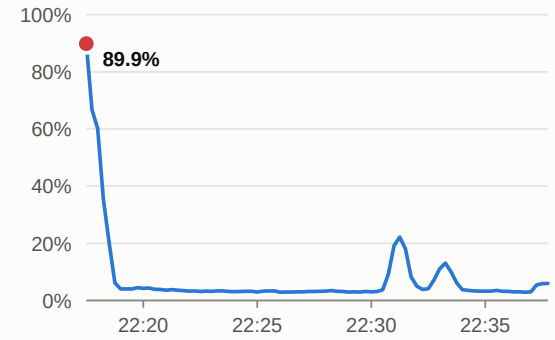
> Each gap is a **label**: "something went wrong here". A model trained on the readings in the 30 minutes before each gap learns what an outage looks like and can then **predict the next one**.

WHY THIS DATA WORKS FOR IT

- The gaps line up across every panel, so they are real events, not one broken chart.
- Labelled incidents are usually the hardest part of outage prediction. This export already has some.
- Logging each incident and deploy from now on gives the model reliable labels to train on.

05 · USE CASE 4 · EVENT CLASSIFICATION

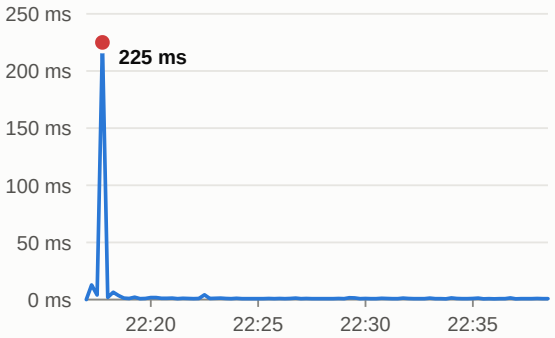
What a planned restart looks like



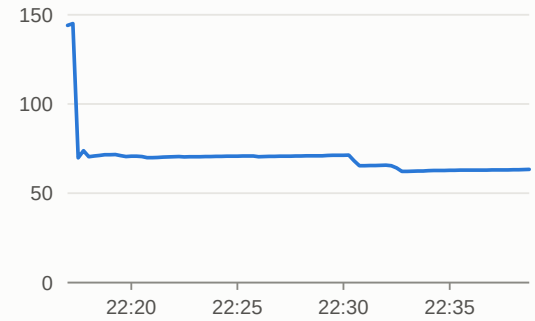
Host CPU busy (%)



Host RAM used (%)



Node.js event loop lag (ms)



Node.js process memory, RSS (MiB)

WHAT THE CHART SHOWS

- Host CPU starts at 89.9% and is down to normal within a few minutes.
- Host RAM climbs from 63.1% to 84.2% in 30 seconds as containers load.
- Event loop lag spikes to 225 ms once, then stays around 0.9 ms.
- Node memory starts at 144 MiB and settles at 63 MiB.
- The service stays UP throughout.

THE IDEA

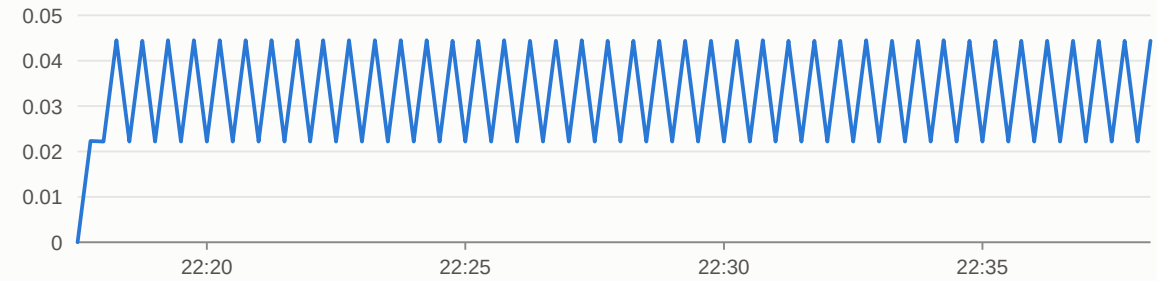
> A **classification model** learns this pattern as "planned restart" and the opposite pattern (service DOWN, errors, no recovery) as "real failure", so on-call staff are only woken for real problems.

WHY THIS DATA WORKS FOR IT

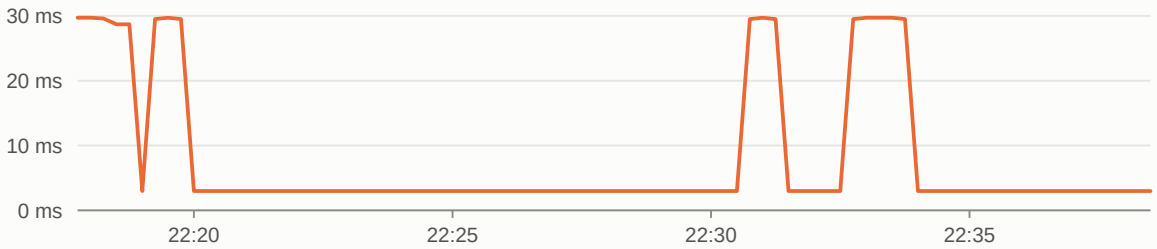
- The four panels change in a fixed order with a clear shape, which is easy to recognise.
- 15-second sampling captures the whole start-up in detail.
- Fewer false alarms make the alerts that remain far more trusted.

06 · USE CASE 5 · CAPACITY PLANNING

A lot of server, very little traffic



Requests per second (only route seen: /api/health)



API P99 latency (ms)



Service availability: green = UP at every 15-second check

WHAT THE CHART SHOWS

- The only traffic is a health check about every 30–45 seconds (about 0.033 requests per second). Every response is HTTP 200.
- P99 latency is mostly 2.97 ms, with jumps to about 30 ms at start-up (22:17–22:19) and twice more (22:31, 22:33).
- Host CPU averages only 3.6%, yet containers hold about 11.9 GiB of RAM.
- The iMOPS Node process itself uses only about 63 MiB, so other containers use most of the memory.

THE IDEA

A **capacity model** links traffic to CPU, memory and latency, then answers: is the server too big or too small, how much traffic can it take, and which container should move elsewhere.

WHY THIS DATA WORKS FOR IT

- The server has spare CPU but is short of memory, a mismatch a model can measure in money.
- Request rate and latency are both recorded, so cost per request can be learned once real traffic arrives.
- It supports cost savings and safe scaling with numbers instead of guesses.

07 · SUMMARY

Five use cases at a glance

	USE CASE	MODEL TYPE	SIGNAL IN THIS DATA	VALUE
1	**Predict when the disk will be full**	Forecast (linear trend, Prophet, ARIMA)	Disk +0.09 pts/hour, 90.3% now	Fix it days before it breaks
2	**Spot unusual behaviour**	Anomaly detection (Isolation Forest, z-score)	Stable baseline, 667% CPU spike	Catch problems without hand-written alert rules
3	**Warn before an outage**	Classifier (XGBoost, LSTM)	RAM ~86% for hours, 4 data gaps	Act 15–30 min before the service goes down
4	**Restart or real failure?**	Pattern classification / clustering	Clear 22:17 restart signature	Fewer false alarms for on-call staff
5	**Plan capacity**	Regression + forecasting	CPU 3.6% vs ~12 GiB RAM used	Right-size the server, save cost